WISEConv: Worklist-Driven Masked Convolution for Low-Latency Vision Inference

Anonymous Author(s)

Abstract

Masked convolution promises to reduce vision-model inference latency by computing almost exclusively the positions an upstream policy marks active, yet existing paths sacrifice either throughput or useful-compute ratio and fail to translate sparsity into proportional latency reduction. We present WISEConv, a masked-convolution runtime built on the observation that output coordinates form the interface between position selection and the regular reduction datapath in the dense tiled pipeline. WISEConv replaces the dense tiled pipeline's contiguous coordinate source with an active-output worklist, restricting computation to almost exclusively active positions while retaining dense tensor layouts and the regular reduction datapath. To keep worklist construction cheap and sustain throughput as the number of active outputs changes per frame, WISEConv fuses mask propagation with tile-local compaction, reuses coordinate metadata across compatible operators, and adapts its compute decomposition without host synchronization, securing both a high useful-compute ratio and high throughput. We evaluate WISEConv on four perception models across six GPU platforms: four discrete GPUs and two Jetson platforms. WISEConv achieves a 1.58× geometric-mean speedup over the fastest competing path per model-platform pair, reducing full-model latency by 25.0–52.8%. On the Jetson platforms, it also reduces per-frame board energy by 29.9–67.8% relative to dense execution.

Keywords: Sparse convolution, dynamic spatial sparsity, GPU runtime, deep learning systems, efficient inference

1 Introduction

Dense convolution dominates per-frame inference latency in many latency-critical vision tasks [3, 10, 40], yet on a given input much of that computation is spatially redundant. A common remedy is masked convolution: an upstream policy marks spatially active positions on the dense feature grid, and a dedicated sparse convolution operator updates only these positions in the dense output. The mask may derive from event-camera increments [8, 24], temporal differences [17, 30], or learned spatial gates [22, 37]. These mask sources promise large reductions in convolution work, yet existing masked-convolution systems [8, 22, 30, 37] rarely translate the sparsity into proportional latency reduction.

Dense GPU convolution obtains high *throughput* (operations per second) from coordinate-driven, tiled execution. Each tile enumerates a contiguous block of output coordinates; those coordinates fix each output position's receptive field and write location, while the reduction over channels and kernel offsets follows a regular datapath with predictable memory access. In masked convolution, reducing latency proportionally requires sustaining this throughput while achieving a high *useful-compute ratio* (the fraction of issued operations that produce needed outputs), but the per-frame position selection imposed by the mask conflicts with the contiguous coordinate assumption that sustains this throughput, placing the two requirements in tension.

Existing systems follow one of two execution paths, retaining tiled execution for high throughput or eliminating wasted work for a high useful-compute ratio, but not both. *Tile skipping* [8, 17, 30, 32] retains the dense pipeline and suppresses a tile only when all its outputs are inactive, so a single active position triggers computation for the entire tile. *Gather-scatter* [22, 37, 41] instead extracts exactly the active positions into a packed representation, computes, and scatters the results back, incurring extraction, irregular access, and writeback overhead that damages throughput.

Neither path converts sparsity into proportional latency reduction (§2.2). In our evaluation, under a temporal difference mask policy [30], 52.2% of YOLOv8n [19]'s convolution work is unnecessary, yet tile skipping [30] incurs 24.5% higher latency than dense with a useful-compute ratio of only 66.7%; gather-scatter [5] lifts the useful-compute ratio to near one but at 4.5× the latency of dense. Even on the sparser FireFlowNet [28] under event-camera increments [8], where 74.7% of work is unnecessary and tile skipping does cut latency by 42.0%, it still wastes 30.2% of its issued operations; gather-scatter sustains only 21.3% of tile skipping's throughput, at 1.9× the latency of dense.

Our key insight is that output coordinates form the interface between position selection and the reduction datapath in tiled execution. Replacing the contiguous coordinate source with an active-output worklist changes only which positions are scheduled, while preserving the dense tensor layouts and regular reduction datapath that sustain throughput. The worklist interface makes it possible to compute almost exclusively required outputs without abandoning the dense pipeline's throughput potential.

Realizing this potential requires solving two challenges. First, *worklist construction cost*: construction scans the mask, so its cost scales with the layer's spatial grid, not with the active count or channel widths, while the compute it enables shrinks with both. Construction's share of latency therefore

grows as activity falls and channels thin, and accumulates across operators when every layer rebuilds. Second, *worklist consumption efficiency*: the worklist's order and length both affect its consumption throughput. An unordered worklist scatters the input neighborhoods that adjacent work touches, lowering locality; its length falls below the dense grid and shifts with layer and input, so a naive fixed decomposition cannot keep the GPU full, and because the length is device-resident, any host-side decision that depends on it costs a synchronization on the per-frame path. Both effects sharpen as activity falls.

We present WISEConv (**W**orklist-dr**I**ven ma**S**k**E**d **Conv**olution), a masked-convolution runtime that fuses position-level selectivity into the dense tiled pipeline, enabling both a high useful-compute ratio and high throughput by co-designing worklist construction and consumption. To reduce worklist construction cost, a single mask-space pass fuses output-mask propagation with per-tile compaction, operating on only mask and coordinate metadata; compatible operators reuse valid metadata instead of reconstructing it.

To sustain worklist consumption efficiency, construction emits contiguous per-tile segments ordered by the dense tile traversal, giving the compute stage tile-local structure without global sorting. Because the worklist changes the number of output-coordinate rows while each row's tensor layout and reduction datapath remain unchanged, WISEConv launches a persistent grid that serves any worklist length, with idle thread blocks absorbing reduction work from under-subscribed tiles. The kernel reads the device-resident worklist length and indexes a precomputed table and decomposes the workload accordingly, sustaining GPU utilization across varying worklist lengths without host synchronization.

This paper makes the following contributions:

- We characterize existing masked-convolution execution paths along two axes, useful-compute ratio and throughput, exposing why they do not translate upstream sparsity into proportional latency reduction.
- We design WISEConv, which fuses position-level selectivity into the dense tiled convolution pipeline and co-designs worklist construction and consumption to secure both axes jointly.
- We evaluate WISEConv on four perception models across six GPU platforms: four discrete GPUs and two Jetson platforms. WISEConv is the fastest path in all 24 workload-platform pairs, achieving a 1.58× geometric-mean speedup over the fastest evaluated competing path in each pair (25.0–52.8% lower full-model latency) and reducing per-frame board energy by 29.9–67.8% relative to dense execution on the Jetson platforms.

2 Background and Motivation

2.1 Masked Convolution

Convolution runs inside the perception-action loop of many vision applications [3, 34, 38, 40, 44], where per-frame inference latency bounds system responsiveness [9–11, 18]. To cut this latency, masked convolution exploits dynamic spatial sparsity: an upstream policy emits a spatial mask, and the operator computes only the marked positions. The mask comes from sources including event-camera increments [8, 24], temporal residuals between video frames [4, 17, 29, 30, 39], or learned input-dependent gates [13, 22, 37, 41]. These sources differ in sensing modality and policy, but all expose the same dynamic spatial sparsity on a dense 2D feature grid, so a single masked-convolution runtime serves them all.

Masked convolution retains dense tensor layouts and arithmetic, restricting only which output coordinates are computed. It operates on an input feature tensor $\mathbf{x} \in \mathbb{R}^{H_{in} \times W_{in} \times C_{in}}$, a weight tensor $\mathbf{f} \in \mathbb{R}^{k \times k \times C_{in} \times C_{out}}$ of kernel size k, and an output $\mathbf{y} \in \mathbb{R}^{H_{out} \times W_{out} \times C_{out}}$ on a coordinate grid $\mathcal{G} = \{1, \ldots, H_{out} W_{out}\}$, where each coordinate $p \in \mathcal{G}$ indexes a spatial position (h, w) and has a $k \times k$ receptive field $\mathcal{N}(p)$ in $\mathbf{x}$. An upstream policy marks the coordinates to compute in an output mask $M_{out} \in \{0,1\}^{H_{out} \times W_{out}}$. Spatial gating predicts M_{out} directly; event and temporal policies instead supply an input mask $M_{in} \in \{0,1\}^{H_{in} \times W_{in}}$ whose active positions propagate their influence along the receptive field, so p stays active whenever any input in $\mathcal{N}(p)$ is active,

$$M_{out}[p] = \begin{cases} 1, & \exists\, r \in \mathcal{N}(p) \text{ s.t. } M_{in}[r] = 1, \\ 0, & \text{otherwise.} \end{cases} \tag{1}$$

M_{out} thus identifies the output positions that the execution path is required to compute, each by aggregating $\mathbf{x}$ over $\mathcal{N}(p)$ with the weights $\mathbf{f}$.

An execution path issues computation over a sequence of position slots C, where each slot is drawn from $\mathcal{G} \cup \{\perp\}$; $\perp$ denotes a padding slot that carries no output coordinate. The path must satisfy the *coverage contract*: every active coordinate is included, $\{p : M_{out}[p] = 1\} \subseteq \{c \in C : c \neq \perp\}$. For each non-$\perp$ slot $p \in C$ it computes

$$\mathbf{y}[p] = \sum_{i=1}^{k^2} \mathbf{x}[\mathcal{N}_i(p)]\, \mathbf{f}_i, \tag{2}$$

where $\mathcal{N}_i(p)$ is the i-th position in $\mathcal{N}(p)$ and $\mathbf{f}_i \in \mathbb{R}^{C_{in} \times C_{out}}$ the corresponding weight slice. How unrequested positions are treated is policy-specific: event and temporal execution retains prior state, while learned gating merges results into a residual. Coverage is therefore the operator-level correctness condition; the policy determines the treatment of all other positions. Beyond coverage, a path may issue inactive or $\perp$ slots; dense convolution is the extreme where $\{c \in C : c \neq \perp\} = \mathcal{G}$. The policy fixes the accuracy and required work.

2.2 The Sparsity-to-Latency Gap

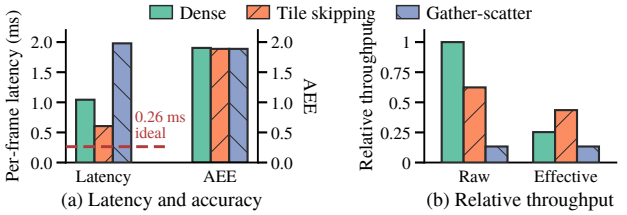


Figure 1. FireFlowNet [28] using the Ev-Conv [8] event-driven mask policy on MVSEC [45] and RTX 3080, comparing cuDNN (Dense), DeltaCNN tile skipping [30], and MMCV's native gather-scatter path [5]. (a) Mean full-model per-frame latency and average endpoint error (AEE; lower is better). The dashed line marks dense latency scaled by the active ratio aggregated across layers; the sparse paths use the same masks and produce nearly identical AEE. (b) Raw throughput T and effective throughput T_{eff}, each normalized to dense raw throughput. The dense useful-compute ratio equals the active ratio set by the Ev-Conv mask policy ($\alpha \approx 25.3\%$).

Existing systems execute a mask along one of two paths. *Tile skipping* [8, 17, 23, 29, 32, 39] (Figure 9b) inherits the dense pipeline's tile-based computation without additional metadata passes, skipping a tile only when all its coordinates are inactive and otherwise computing the whole tile; DeltaCNN [30] fuses a selective path into the kernel, taken when a tile is estimated to be mostly empty, to cut wasted work. *Gather-scatter* [22, 41] (Figure 9c) instead abandons the dense pipeline: it extracts exactly the active coordinates into a packed representation, runs a separate GEMM, and scatters the results back, removing the inactive arithmetic but adding extraction, irregular access, and writeback overhead; DynConv [37] adopts a model structure suited to gather-scatter, where one gather feeds several consecutive layers before a single scatter, amortizing that overhead.

A policy fixes the active ratio α, defined as the fraction of output-grid positions marked active, whereas a concrete execution path determines the useful-compute ratio η, defined as the fraction of issued slots that correspond to required outputs:

$$\alpha = \frac{\|M_{out}\|_0}{|\mathcal{G}|}, \quad \eta = \frac{\|M_{out}\|_0}{|C|}. \tag{3}$$

For $|C| > 0$, an ideal path achieves $\eta = 1$ by issuing only required outputs; dense convolution, which issues the entire grid, has $\eta \approx \alpha$. An execution path also determines its throughput T, which combines with η to yield the effective throughput T_{eff}:

$$T = \frac{2|C|k^2 C_{in} C_{out}}{t}, \quad T_{\text{eff}} = \eta T. \tag{4}$$

where t denotes the latency of the execution path, including auxiliary mask and coordinate processing, k is the convolution kernel size, and C_{in} and C_{out} are the input and output channel counts of the masked convolution layer, respectively. The factor of two counts each multiply-accumulate as two FLOPs. Consequently, T_{eff} directly characterizes the processing rate of required outputs and therefore determines the realized latency.

Figure 1 traces this on RTX 3080 for FireFlowNet [28] under the Ev-Conv [8] mask policy. Event cameras expose microsecond-scale changes, and Ev-Conv evaluates optical flow on event-window inputs updated at 1 kHz. The policy marks 74.7% of the convolution work unnecessary, and the sparse paths discard it without reducing accuracy. Scaling dense latency by the fraction of convolution work that remains gives a 0.263 ms ideal proportional-scaling reference (Fig. 1a, dashed). Neither path approaches it: tile skipping (DeltaCNN [30]) reaches 0.605 ms, still 2.30× the reference, and gather-scatter (MMCV [5]) runs at 1.979 ms, 1.90× the latency of dense convolution itself. The gap traces to low effective throughput on both paths. Tile skipping holds throughput at 62.4% of dense but at a useful-compute ratio of only 69.8%; gather-scatter lifts the useful-compute ratio to one, yet extraction and data movement collapse its throughput to 13.3% of dense (Fig. 1b). Neither path efficiently converts the GPU's capability into effective throughput.

2.3 Distinction from Related Sparse Execution

Other related sparse-execution work differs from WISEConv in representation, operator, sparsity type, or platform. 3D sparse-convolution systems [7, 16, 33, 35, 36, 42] operate on sparse point-cloud coordinates and construct coordinate maps for irregular input-to-output relations; WISEConv instead consumes a runtime mask over a dense 2D grid while retaining dense tensor layouts. Sparse matrix multiplication [12, 14, 31] accelerates sparse-graph and pruned-weight matrix products, where sparsity is structural rather than a dynamic spatial mask over a convolution grid. Structured weight pruning (e.g., 2:4 sparsity [26]) exploits fixed patterns in model weights. Sparse-format compilers [21, 43] expose sparsity through explicit tensor formats, whereas WISEConv retains dense layouts and uses a transient per-frame mask only to select output coordinates. DPACS [15] resolves similar dynamic spatial sparsity in CNN feature maps, but on a custom accelerator instead of a commodity GPU.

Some 3D systems [33, 36] compute through coordinate-indexed fused GEMM [1], where an index array derived from coordinate maps redirects the position dimension of a tiled GEMM. It seemingly coincides with WISEConv's goal, but 3D sparse convolution requires explicit coordinate metadata because its tensors are represented as sparse coordinate sets. Constructing the index array incurs hashing, sorting, and global scans that frequently dominate per-layer latency over the GEMM itself [36, 42], making construction cost one of the central research problems in the 3D community; thus directly introducing coordinate indirection into 2D convolution which already tiles densely appears unattractive. 2D

masked convolution, however, presents a structurally simpler mapping problem: the output grid is regular, the mask is a bitmap, and each coordinate's receptive-field addresses follow from fixed grid arithmetic, so the coordinate-map machinery of 3D systems is unnecessary. WISEConv exploits this simpler mapping by replacing the coordinate source without altering the tiled reduction.

3 Characterizing Worklist-Driven Convolution

The sparsity-to-latency gap (§2.2) leaves open whether a worklist-driven convolution closes it and what it takes. Every such operator has two stages: a construction stage that turns the mask into a worklist, and a compute stage that runs the dense implicit GEMM with its row coordinates read from the worklist instead of enumerated. We characterize the two stages separately, so that each finding belongs to the execution model rather than to a particular implementation of it. For the compute stage we ask how close its latency comes to proportional and what that requires of the kernel configuration (§3.1), why irregular rows leave the per-tile throughput intact (§3.2), and how much the order of the worklist matters (§3.3); for the construction stage, what it costs relative to the computation it enables (§3.4).

Methodology. **Compute stage.** We implement the worklist GEMM following CUTLASS's gather implicit-GEMM convolution, with gather indices replaced by worklist entries: $\lceil n_W/B_M \rceil$ tiles, each loading B_M worklist rows and reducing over the k^2 taps and C_{in} channels exactly as the dense kernel does. A persistent grid reads n_W on the device, so no length reaches the host. We instantiate the kernel over the configuration space a dense autotuner searches: tile shape (B_M, B_N, B_K), warp layout, pipeline depth, and split-K within a CTA or across CTAs; FP16 uses `mma.sync` tensor cores and FP32 a SIMT kernel of the same structure. Each configuration is timed in isolation inside CUDA graphs on worklists that are given, not constructed. **Construction stage.** We time the mask-to-worklist pass of each layer on its own (§3.4). **Reference.** Dense latency t_{dense} is the faster of the same kernel on the full grid and cuDNN's non-Winograd algorithms; Winograd changes the arithmetic, and a masked layer cannot use it. **Workloads.** The convolution layers of YOLOv8n and YOLOv8m on WildTrack, FireFlowNet on MVSEC, and DynConv on MPII (Table 2), driven by synthetic worklists of a given active ratio or by the worklists their mask policies produce on real inputs (74,844 layer-frame samples for YOLOv8n alone). **Platforms.** RTX 3080 (68 SMs) and Jetson AGX Orin (16 SMs) at locked clocks.

3.1 Proportionality and the Configuration It Requires

The per-length best configuration is nearly affine. A worklist kernel issues only active outputs ($\eta \approx 1$, apart from the padding of the last tile), so proportional latency, $t(\alpha) = \alpha\, t_{\text{dense}}$, is equivalent to keeping the dense throughput. We first grant the kernel its best case and pick the fastest configuration for every worklist length. Its latency is affine in α rather than proportional (Figure 2a): $t/t_{\text{dense}} = 0.794\alpha + 0.258$ for FP32 and $0.643\alpha + 0.417$ for FP16 on RTX 3080, and $0.917\alpha + 0.085$ and $0.830\alpha + 0.267$ on AGX Orin, with a median per-layer R^2 of 0.97–0.995. Each line meets dense latency at $\alpha = 1$ (slope and intercept sum to about one), so the whole departure from proportionality is the intercept b, a latency every layer pays regardless of how many positions are active. Table 1 breaks b down with hardware counters: it is launch and synchronization latency, not memory traffic. An empty kernel spends 92% of its stall cycles on constant-cache misses (the kernel parameters are cold at every launch), and a single tile adds a dependent chain of barriers, constant-cache misses, and, with split-K, memory fences, with memory-latency stalls at only 6–27% and the data in L2. The intercept is 31% (FP32) and 47% (FP16) of a median layer's dense time on RTX 3080 and 10% and 33% on AGX Orin: a layer comes closer to proportional the larger it is relative to the device.

Fig. char-affine: (a) normalized latency vs. α (log), 3080 / AGX, FP32 / FP16: diagonal, per-length best + affine fit, dense-tuned. (b) real layer-frame samples: per-length best vs. distribution-calibrated, two affine fits.

Figure 2. Convolution latency normalized to dense, t/t_{dense}, summed over the layers of YOLOv8n. (a) Synthetic worklists of active ratio α: the fastest configuration at each worklist length (per-length best) follows an affine line close to the proportional diagonal; the configuration a dense autotuner picks flattens at low α. (b) Worklists of real frames on four workloads: one configuration per layer, calibrated on separate frames of the same workload, traces the per-length best's affine line with an intercept larger by 0.01–0.03.

Table 1. Composition of the intercept b on a representative layer: latency and dominant stall reasons of the worklist kernel with no active position, one tile, and the full grid (Nsight Compute, RTX 3080 and AGX Orin).

Table char-intercept: rows $n_W = 0$, one tile, $\alpha = 1$; columns latency (μs), SM-active time, stall shares (constant-cache miss, barrier, fixed-latency dependency, membar, long scoreboard), DRAM bytes; both GPUs.

Fig. char-persm: slowdown of a fixed configuration over the per-length best vs. work per SM (log), dense-tuned and activity-agnostic single configuration, 3080 and AGX overlaid.

Figure 3. Slowdown of a fixed configuration over the per-length best across the synthetic sweep of Figure 2a, binned by the work per SM of the fixed configuration (its tiles times its K splits, divided by the SM count). Both GPUs fall on one curve: a configuration is near-optimal only in the regime of work per SM it suits.

Fig. char-mechanism: CDF over layers of the expected per-frame gain of per-length selection over the layer's best single configuration (μs); vertical lines: per-layer cost of host readback, updatable node, conditional node; markers: realized gain of in-kernel dispatch and of lagged graph selection.

Figure 4. Per-layer gain that selecting the configuration for every worklist length could deliver over the best single configuration of the layer (CDF over the layers of four workloads, RTX 3080 and AGX Orin), against what each runtime mechanism costs or realizes: host readback and launch, a device-updatable kernel node, and a conditional graph node pay a fixed cost per layer; in-kernel dispatch among fused configurations and a CUDA graph chosen per frame from lagged activity pay almost nothing but realize only part of the gain.

Over the full range of activity, the best configuration changes. No single configuration stays near this line from $\alpha \to 0$ to $\alpha = 1$. The configuration a dense autotuner picks, the fastest at $\alpha = 1$, runs 2.33× (FP32) and 1.64× (FP16) slower than the per-length best when it gives an SM less than a quarter of a tile on RTX 3080, and 4.00× and 1.92× on AGX Orin, and matches the best above four tiles per SM (Figure 3). Large tiles amortize each weight load over many rows; when rows are few, smaller tiles and split-K expose enough independent work to occupy the SMs and shorten the chain each one executes. Picking the configuration with the lowest expected latency over α uniform on $(0, 1]$ only moves the loss: it is still 1.61× and 1.39× slower below a quarter of a tile per SM on RTX 3080, and 11.6–13.5% slower at $\alpha = 1$. A layer therefore needs a configuration that depends on its activity. Whether it needs one per worklist length depends on how much of this range a layer actually visits, which we examine next.

Selecting per length gains little, and every mechanism pays for it. Against the best single configuration for the layer's own frames, choosing per worklist length saves a median layer only 0.6 μs per frame on RTX 3080 and 1.5 μs on AGX Orin (Figure 4). Capturing this gain at run time is costly, because the length that drives the choice exists only on the device and changes at every layer of every frame. Mechanisms that decide per layer pay a fixed cost per layer: reading the length back serializes each layer behind a host synchronization, a device-updatable kernel node rewrites the next launch at 0.9 μs on RTX 3080 and 2.2–3.3 μs on AGX Orin, and a conditional graph node selects a subgraph at 17–19 μs on AGX Orin; only X% of the layers gain more than the cheapest of them costs. Mechanisms that avoid the per-layer cost give up part of the gain. Fusing the candidate configurations into one kernel and branching inside it decides for free, but the fused kernel carries the register and code footprint of all its paths and realizes 30–60% of the gain. Choosing one of a few pre-captured CUDA graphs per frame from the activity of past frames decides on the host without synchronization, but a frame-level prediction cannot follow layers whose activity varies independently, and it realizes 50% of the gain on YOLOv8n and 10% on DynConv.

One calibrated configuration per layer reaches the same line. The range a layer visits is far narrower than $(0, 1]$, so a configuration suited to that range suffices. We calibrate one configuration per layer to its activity distribution, taking the configuration with the lowest mean latency on calibration frames, and evaluate it on separate frames of the same workload (a held-out camera for WildTrack, held-out clips otherwise). On real worklists its affine fit shares the per-length best's slope, and its intercept is larger by only 0.01–0.03 of dense time on both GPUs and precisions (Figure 2b); summed over a model it is within 2–4% of the per-length best on YOLOv8n, YOLOv8m, and FireFlowNet and within 1% on DynConv. DynConv is the hardest case for a single choice: its learned gates vary a layer's active ratio by up to 9× between the tenth and ninetieth percentile frame, nearly independently of the other layers. The configuration a dense autotuner picks, in contrast, is 42–94% slower than the calibrated one on the YOLO models: what matters is calibrating to the activity a layer sees, not adapting to each frame of it.

Why one configuration suffices. A configuration's latency has two regimes in the worklist length (Figure 5a). While its tiles fit in one wave of co-resident CTAs, every CTA runs at most one tile and the latency is the single-tile chain plus the launch, nearly flat in n_W; the best configuration here is the one with the shortest chain at the layer's shape, which does not move with n_W. Beyond one wave the latency is a staircase, $t_q(n) \approx \lceil \text{tiles}_q(n)/W_q \rceil \, t_{\text{tile},q} + t_{0,q}$ for W_q co-resident CTAs, and configurations with different tile shapes place their steps at different lengths: the per-length

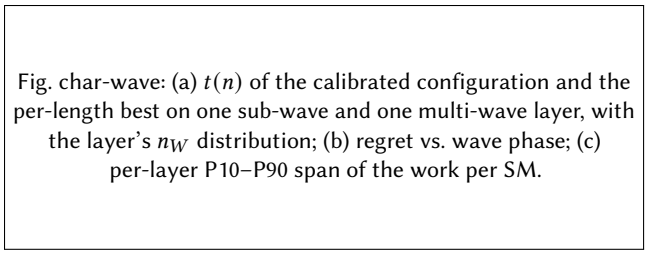


Figure 5. Why one configuration suffices. (a) Latency against worklist length for a layer whose worklists fill less than one wave and for a layer that spans several, with the layer's distribution of lengths shaded: below one wave the latency is nearly flat, above it a staircase. (b) Regret of the calibrated configuration over the per-length best against the calibrated configuration's wave phase: in the multi-wave regime it concentrates just past a wave boundary. (c) Span of each layer's work per SM between its tenth and ninetieth percentile frame.

best beats a fixed configuration mainly just past one of its steps, where a partial wave costs a full tile time (Figure 5b). These steps account for 70% of the per-length gain; the rest comes from layers whose lengths straddle the two regimes. Most layers do not: between its tenth and ninetieth percentile frame, a median layer's work per SM spans a factor of 4 (Figure 5c), and X% of the layers stay on one side of a wave throughout. A layer thus spends nearly all its frames in one regime, where one configuration is best up to the steps.

The distribution, not the frame, sets the configuration. Calibration does depend on the distribution it sees. Scaling every layer's worklist lengths by a factor between 1/4 and 4 makes the configuration calibrated at the original scale 3–24% slower than one re-calibrated at the new scale, which in turn stays within 1–3% of the per-length best. Real shifts behave the same way: a configuration calibrated on one MVSEC sequence loses X% on a sequence with 4× its activity. A deployment must therefore calibrate to the activity it will see and recalibrate when that activity changes substantially, but within a distribution it needs no per-frame adaptation.

Insight 1. *With the configuration chosen for every worklist length, a worklist-driven implicit GEMM is nearly affine in the active ratio, but that choice gains little over one configuration calibrated to a layer's activity distribution, and every runtime mechanism either costs more per layer or realizes only part of it. The calibrated configuration reaches the same line with a slightly larger intercept, because a layer's work per SM stays in one regime across frames, where per-length selection only exploits wave-quantization steps.*

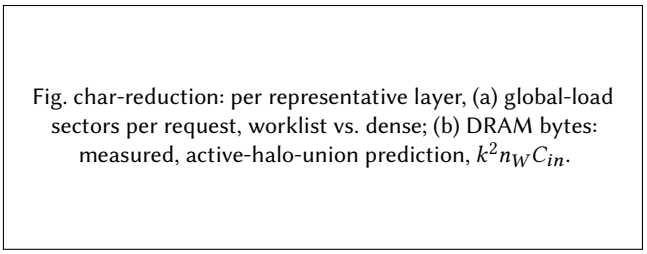


Figure 6. Per-tile memory behavior of the worklist kernel on seven representative YOLOv8n layers (RTX 3080 shown; AGX Orin is the same). (a) Sectors per global-load request stay close to the dense kernel's, except for the stem, whose 32-byte rows cannot fill a request. (b) DRAM traffic equals the union of the active positions' input halos plus the weights, not the $k^2 n_W C_{in}$ a gather-scatter path moves.

The compute stage (§5.3) therefore provides a configuration space rich in fine decompositions, calibrates one configuration per layer to the deployment's activity, and has the kernel consume the device-resident worklist length directly.

3.2 Irregularity Stays Outside the Reduction Loop

The affine line meets dense latency at $\alpha = 1$, so an active position costs the worklist kernel what it costs the dense kernel. The reason lies in where the worklist enters the kernel. An implicit GEMM draws on three levels of reuse: contiguous channels within a row (memory coalescing), a weight tile shared by the B_M rows of a CTA (arithmetic intensity), and overlapping receptive fields of neighboring outputs (L1 and L2 hits). The worklist changes only the base address of each row. A tile issues B_M index loads in its prologue, ahead of the multistage pipeline, and the reduction loop over k^2 taps and channel slices keeps the dense kernel's affine addressing. The first two levels of reuse are therefore untouched. Counters confirm it: a worklist tile executes the same instructions as a dense tile (within 0.5%, with identical FMA, tensor, and global-load counts), and global loads keep 12.8–14.3 sectors per request against 14.5–15.5 in the dense kernel for FP32 (8.6–12.7 against 11.0–15.4 for FP16; Figure 6a). The one exception is the predicted boundary: the stem layer has eight padded input channels, a 32-byte row that fills only 3.7 sectors per request in both kernels. DRAM traffic is set by the union of the active halos: measured bytes are 1.00–1.11× that prediction, while gathering each position's receptive field separately would move 2–7× more on 3×3 layers (Figure 6b). Point-cloud sparse convolution differs exactly here: its input is sparse too, so every reduction step depends on a rulebook lookup.

3.3 Worklist Order Barely Matters

The third level of reuse, overlapping receptive fields, depends on which positions share a CTA and therefore on the order

Fig. char-order: distribution over layer-frame samples of latency relative to the construction order, for sorted / shuffled / uniform worklists, 3080 and AGX; inset: DRAM, L1-hit, L1-to-L2 traffic on representative layers.

Figure 7. Latency of four worklists with the same length and tile on real frames, relative to the order construction emits: sorted in global dense order, shuffled, and uniform (the same count of positions spread uniformly, removing the mask's spatial clustering). Inset: counters on representative layers; order changes L1 hits, never DRAM bytes.

of the worklist. It matters little (Figure 7). Across 3,560 layer-frame samples, a globally sorted worklist runs at 0.999× the latency of the order construction emits (median) and a shuffled one at 1.004–1.007×; summed over the model, every variant is within 2.2% on RTX 3080. Counters show why: the construction order, sorted, and shuffled worklists move identical DRAM bytes, and order only shifts reuse between L1 (6–46% hits for the construction order, 3–26% shuffled) and L2, which absorbs the rest. Order becomes visible only where L2 bandwidth is scarce relative to the work: on AGX Orin, shuffling slows the early layers with many positions and little computation each by 1.10–1.33×, because the extra L1-to-L2 traffic (7–70% more) surfaces on a GPU with little L2 bandwidth. Even there, a tile-local order recovers all of the global order's benefit (sorted against construction order: 0.993–0.999×).

Insight 2. *Worklist order never changes DRAM traffic, which the union of the active halos sets; it only shifts reuse between L1 and L2. A random order costs about 1% except on low-intensity layers of a GPU with little L2 bandwidth, and a tile-local order already recovers everything a global order would.*

Construction therefore needs to preserve only tile-local order (§5.1).

3.4 Construction Cost Does Not Shrink with Activity

A worklist must exist before the computation it enables, and building it scans the mask grid of the layer's geometry, whose size does not depend on activity. Its cost is therefore nearly constant per layer: 1.7–3.2 μs on RTX 3080 and 4.2–23.6 μs on AGX Orin, dominated by a fixed launch cost (about 1.8 μs and 4 μs) and visible mask scanning only on the 640×640 stem. The computation it enables shrinks with activity and channel count. Over a YOLOv8n frame, 52 constructions (the other 11 layers reuse their input's worklist) take 93 μs on RTX 3080 against 397 μs (FP32) and 232 μs (FP16) of computation, a median share of 19% and 29%; on AGX Orin they take

Fig. char-construct: per-frame construction share of convolution time vs. frame α, 3080 / AGX, FP32 / FP16; inset: per-layer construction vs. compute by C_{in}.

Figure 8. Share of convolution time spent building worklists, per YOLOv8n frame, against the frame's active ratio. Construction costs a near-constant time per layer, so its share rises as activity falls and as tensor cores shorten the compute.

258 μs, 11% and 27%. The share tracks the frame's activity (correlation −0.94 and −0.96 on RTX 3080, −0.81 and −0.93 on AGX Orin): it reaches 24% and 33% in the least active quarter of frames on RTX 3080 and falls to 12% and 23% in the most active quarter (Figure 8). Thin layers pay the most, because the computation scales with C_{in} and the construction does not (construction reaches 0.48 of compute below 32 input channels against 0.36 above 96).

Insight 3. *Building a worklist costs a near-constant time per layer, independent of activity and channel count, so its share of the convolution grows exactly where sparsity is most useful.*

Construction must therefore touch only the mask and be shared wherever layers can use the same worklist (§5.1, §5.2).

4 WISEConv Overview

Dense tiled convolution already consumes a stream of output coordinates: each coordinate determines the receptive-field reads and output write, while its reduction over channels and kernel offsets remains regular. WISEConv exploits this interface by replacing the dense coordinate sequence with a worklist of requested output positions, thereby introducing position-level selectivity into the tiled pipeline. Removing unrequested positions from the schedule raises the useful-compute ratio, but issuing fewer operations alone does not guarantee lower latency. To translate selectivity into latency reduction, the worklist must be inexpensive to construct and must be consumed with both a high useful-compute ratio and high throughput. WISEConv therefore co-designs a construction stage and a compute stage around these interdependent requirements, as Figure 9 summarizes.

Construction stage. To prevent worklist construction from erasing the savings of selective compute, WISEConv confines construction to mask and coordinate metadata (Figure 9d, ①), never reading feature or weight tensors. In one mask-space pass, it fuses the output-mask propagation in

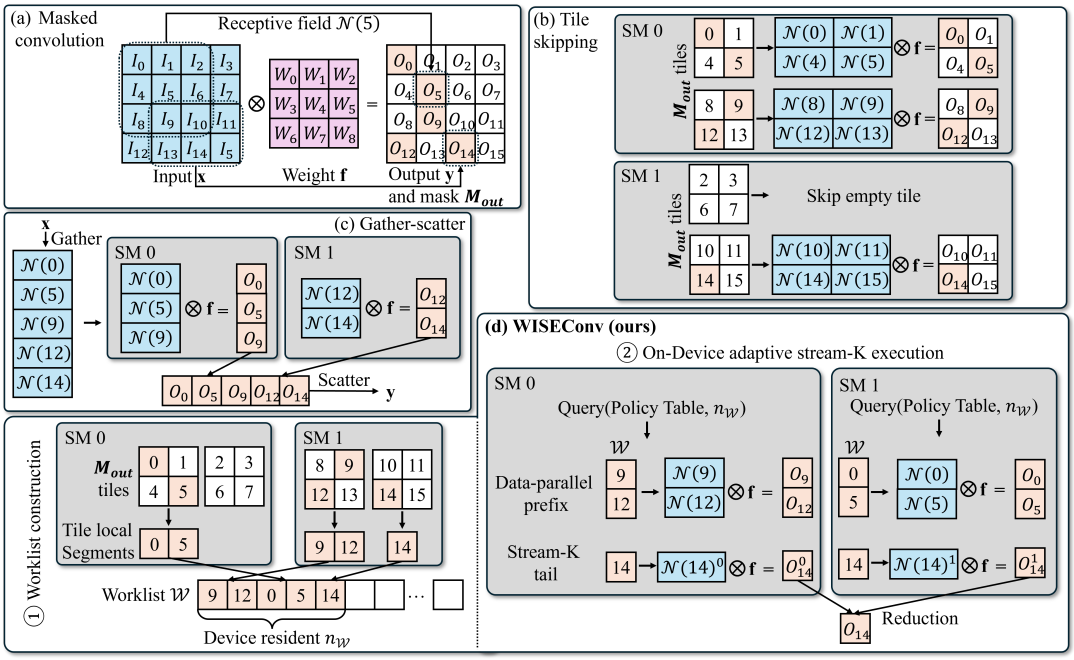


Figure 9. Overview of WISEConv and baseline execution paths on a two-streaming-multiprocessor (SM) example. (a) Masked convolution: M_{out} marks active output positions (colored); the operator computes only these from $\mathbf{x}$ and $\mathbf{f}$. (b) Tile skipping skips only fully inactive tiles; a single active position triggers computation for that tile. (c) Gather-scatter packs exactly the receptive fields of active positions, computes, and scatters results back. (d) WISEConv: ① constructs a tile-local worklist and ② consumes it via on-device adaptive Stream-K execution, where data-parallel tiles complete a full reduction each while Stream-K tail tiles split their reduction across SMs (O^0_{14}, O^1_{14}) and merge the partials.

Eq. 1 with tile-local compaction, producing M_{out} and a worklist $\mathcal{W}$ (the figure shows the compaction step; mask propagation precedes it but is omitted); the requested coordinates from each spatial tile form a segment in local dense-traversal order, preserving spatial grouping without global ordering. WISEConv writes $\mathcal{W}$ into a preallocated buffer large enough for the full output grid. A device-resident counter records $n_{\mathcal{W}} = |\mathcal{W}|$, so construction needs neither runtime allocation nor a host-visible active count.

Because construction cost scales with the spatial grid rather than the active count or channel widths, its share of latency grows as activity falls and accumulates when every layer rebuilds. WISEConv therefore propagates and reuses the resulting mask and worklist metadata across compatible operators instead of reconstructing it, amortizing one construction across the compatible operator sequence. The fused pass lowers the cost of each construction, while cross-operator reuse reduces how often it occurs; together, they address the Worklist Construction Cost challenge.

Compute stage. The compute stage (Figure 9d, ②) replaces only the coordinate source of a dense tiled implicit-GEMM pipeline with the worklist; tensor layouts, weight access, and the per-position channel-and-kernel reduction remain unchanged from the dense path.

The worklist length varies per frame with the input mask: the tile count is unknown before launch and device-resident after construction. Synchronizing it to the host for decomposition selection would insert a per-operator stall on the critical path, yet a fixed decomposition loses efficiency across the resulting range of tile counts. WISEConv builds on Stream-K [27], which addresses GPU underutilization when tiles cannot fill all SMs but assumes the tile count is host-visible at launch. WISEConv designs *on-device adaptive Stream-K execution*: it launches a persistent grid sized to SM residency, and the kernel reads the device-resident worklist length, looks up an offline-calibrated policy table, and selects its decomposition per frame without host involvement. This sustains GPU throughput as the worklist length varies from frame to frame.

Neither stage suffices alone: excessive construction cost, a low useful-compute ratio, or low consumption throughput can each erase the benefit of issuing less convolution work. By controlling all three jointly, WISEConv converts upstream spatial sparsity into end-to-end latency reduction.

5 WISEConv Design

The design follows the worklist from its construction (§5.1) and propagation across compatible operators (§5.2) to its consumption by the convolution pipeline (§5.3).

5.1 Tile-Local Worklist Construction

Constructing a tight schedule already requires a scan of the output grid, but the resulting coordinates must also retain enough spatial structure for efficient consumption. Globally stable compaction preserves the complete dense traversal order but requires a prefix scan or another pass to obtain device-wide offsets; per-coordinate appends avoid that coordination but scatter neighboring positions. Materializing active receptive fields, as gather-scatter does, regularizes the compute input at the cost of feature traffic proportional to the active count and $k^2 C_{in}$. The useful middle ground is therefore a metadata-only schedule with local rather than global ordering.

WISEConv realizes this with contiguous, tile-local worklist segments. Because the number of active positions is not known before the pass, it preallocates a coordinate buffer with capacity $|\mathcal{G}| = H_{out} W_{out}$, the static upper bound on $|\mathcal{W}|$. Algorithm 1 details the resulting single pass. Line 1 initializes the device-resident length counter. Lines 2–7 partition the output grid into $B_\tau \times B_\tau$ construction tiles, derive the corresponding entries of M_{out} using Eq. 1, and compact each tile's active coordinates into a local sequence $\mathcal{P}_\tau$ in dense-traversal order. When a learned gate supplies M_{out} directly [22, 37], Lines 4–5 are omitted and Line 7 compacts the supplied mask. Lines 8–11 count the compacted coordinates, atomically reserve $[\text{base}, \text{base} + n_\tau)$ from the counter, and copy $\mathcal{P}_\tau$ into that interval. After all tiles complete, the reserved intervals collectively store $\mathcal{W}$, and the counter records $n_{\mathcal{W}}$. Because the layer shape fixes the buffer capacity and launch geometry while only the device observes the data-dependent length, construction requires no runtime allocation or host-visible active count.

To relate the worklist to the useful-compute ratio of Eq. 3, we insert its length $n_{\mathcal{W}}$ between the required-position count and the issued-slot count. For $n_{\mathcal{W}} > 0$,

$$\eta = \underbrace{\frac{\|M_{out}\|_0}{n_{\mathcal{W}}}}_{\eta_{\text{construct}}} \cdot \underbrace{\frac{n_{\mathcal{W}}}{|C|}}_{\eta_{\text{compute}}} . \tag{5}$$

We analyze $\eta_{\text{construct}}$ (required positions vs. worklist entries) here and η_{compute} (worklist entries vs. issued slots) in §5.3; the factorization decomposes η only, with all stage costs reflected in throughput T.

Because construction tiles partition the output grid and use disjoint reservations, each active coordinate appears exactly once, so $n_{\mathcal{W}} = \|M_{out}\|_0$. Fresh construction therefore has $\eta_{\text{construct}} = 1$ for a nonempty worklist.

Algorithm 1 Tile-Local Worklist Construction

Require: Input mask M_{in}, output grid $\mathcal{G}$, tile size B_τ
Ensure: Mask M_{out}, worklist $\mathcal{W}$, length $n_{\mathcal{W}} = |\mathcal{W}|$

1: $n_{\mathcal{W}} \leftarrow 0$
2: **for** each $B_\tau \times B_\tau$ spatial tile τ **in parallel do**
3: **for** each position $p \in \tau$ **in parallel do**
4: active$(p) \leftarrow \exists r \in \mathcal{N}(p) : M_{in}[r] = 1$
5: $M_{out}[p] \leftarrow$ active(p)
6: **end for**
7: $\mathcal{P}_\tau \leftarrow$ Compact(τ, M_{out})
8: $n_\tau \leftarrow |\mathcal{P}_\tau|$
9: base $\leftarrow$ AtomicAdd$(n_{\mathcal{W}}, n_\tau)$
10: **for** each $0 \le j < n_\tau$ **in parallel do**
11: $\mathcal{W}[\text{base} + j] \leftarrow \mathcal{P}_\tau[j]$
12: **end for**
13: **end for**

WISEConv thus confines its ordering guarantee to each contiguous segment: coordinates retain the tile's dense traversal, while independently reserved segments provide no cross-boundary locality guarantee. A compute tile of B_M positions (the output-position tile width used by the compute stage, §5.3) may therefore span coordinates from multiple construction segments when segments are shorter than B_M; the preserved locality is intra-segment contiguity, not compute-tile-to-construction-tile alignment. As shown in §7.4, this tile-local order achieves compute-stage throughput comparable to global ordering, making the additional ordering pass an unfavorable end-to-end trade-off.

5.2 Cross-Operator Worklist Reuse

Each construction pass examines all $H_{out} W_{out}$ positions and, when deriving M_{out} from M_{in}, performs $O(k^2)$ mask work per position. This channel-independent grid scan contrasts with required convolution work, which scales with the active ratio, spatial extent, and both channel dimensions. WISEConv therefore amortizes construction by reusing one worklist across each compatible layer sequence.

For layer l, let $\mathcal{G}^l$ denote its output grid, M^l_{out} its output mask, $\mathcal{W}^l$ its worklist, and $n^l_{\mathcal{W}}$ its device-resident length. WISEConv propagates the mask and worklist metadata alongside the feature tensor. Whether two consecutive layers are reuse-compatible depends on their operator types, not on runtime mask values. An operator that preserves the output grid and does not expand the active set guarantees, for any input mask, that $\mathcal{G}^{l+1} = \mathcal{G}^l$ and

$$\{p \in \mathcal{G}^{l+1} \mid M^{l+1}_{out}[p] = 1\} \subseteq \{p \in \mathcal{G}^l \mid M^l_{out}[p] = 1\}. \tag{6}$$

WISEConv checks this condition once at model-conversion time. Because $\mathcal{W}^l$ covers M^l_{out}, Eq. 6 guarantees coverage for M^{l+1}_{out}. WISEConv therefore forwards $\mathcal{W}^{l+1} = \mathcal{W}^l$ and $n^{l+1}_{\mathcal{W}} = n^l_{\mathcal{W}}$ without reconstruction and applies the condition recursively at each subsequent layer.

Equality in Eq. 6 covers same-grid 1×1 convolutions, mask-preserving activations, and inference-time normalization, leaving $\eta_{\text{construct}}$ unchanged. Strict inclusion can result from mask-narrowing operators in the upstream policy, such as the activation-fused update truncation that DeltaCNN [30] uses to increase downstream sparsity. The inherited worklist remains a valid superset, trading a lower $\eta_{\text{construct}}$ for one fewer construction pass without violating coverage. Because compute gates all output writes by the exact mask, positions scheduled but no longer active under the current mask participate in reduction yet produce no store; the superset schedule therefore cannot alter any inactive position's observable state.

A change in the output grid violates the geometry constraint, while receptive-field expansion may violate Eq. 6 by introducing positions outside the upstream mask; either case invokes the construction pass of §5.1 to emit a new tight worklist. A lower $\eta_{\text{construct}}$ alone does not trigger reconstruction because tightening a valid superset would pay another grid scan. Common receptive-field-expanding layers such as 3×3 convolutions still create rebuild points throughout the network, leaving device-resident worklist lengths variable across layers and frames. Compute must therefore handle a dynamic problem size.

5.3 On-Device Adaptive Stream-K Execution

Construction and reuse fix $\eta_{\text{construct}}$; the compute stage controls η_{compute} and throughput T. Dense convolution maps to an implicit GEMM with M output positions, $N = C_{out}$ output channels, and reduction depth $K = k^2 C_{in}$. An output tile covers B_M positions and B_N output channels; the reduction over K proceeds in steps of B_K. In the standard *data-parallel* decomposition, each of the $\lceil M/B_M \rceil \times \lceil N/B_N \rceil$ output tiles is assigned to one thread block, which performs the full K reduction; split-K instead replicates each tile across S_k blocks that each reduce a disjoint K-slice. Because M, N, and K are static, the tile shape (B_M, B_N), reduction step B_K, and split factor can be selected once offline by profiling and keeping the fastest.

In worklist convolution, N and K remain static, but the M dimension becomes $n_{\mathcal{W}}$, the device-resident worklist length written by construction, which varies by an order of magnitude or more across frames of a single layer. The offline assumption breaks: a single configuration cannot serve the full range. Two failure regimes bracket the problem. When activity is high and output tiles are plentiful, K-axis parallelism adds coordination and accumulation overhead without improving occupancy. When activity is low and tiles are scarce, data-parallel execution leaves streaming multiprocessors (SMs) without work because the tile count no longer fills a full *wave* (the set of thread blocks that co-reside across all SMs), referred to as the problem of wave quantization. Adapting between these regimes requires knowing $n_{\mathcal{W}}$; transferring it to the host inserts a per-operator synchronization on the critical path.

WISEConv builds on Stream-K [27], the established solution to wave quantization in dense GEMM. Stream-K launches a *persistent grid* of g_{res} blocks determined by the device's SM occupancy (not the tile count) and partitions all tiles' K-iterations into S_k units consumed by grid stride; partial K-slices fill SMs that would otherwise sit idle after the data-parallel waves complete. Its hybrid variant [1] restricts the K-splitting to the partial tail wave, handling complete waves data-parallel to avoid partial-reduction overhead where it is unnecessary. In worklist convolution the tile count changes every frame, so the wave boundary shifts with it. This shifting boundary is the operating point of WISEConv's *on-device adaptive Stream-K execution*: the compute kernel reads the exact worklist length on the device, determines the current wave boundary, and selects between data-parallel, Stream-K, and hybrid Stream-K decomposition accordingly, without host involvement.

WISEConv fixes the tile configuration $\hat{q}^l = (B_M, B_N, B_K)$ per layer at deployment and adapts only the decomposition mode on-device. What changes per frame is how the K reduction across output tiles is assigned to thread blocks, selected from three modes:

- *Data-parallel*: each block owns a complete output tile and reduces the full K range.
- *Stream-K*: each output tile's K-iterations are partitioned into S_k units; the persistent grid consumes all units by grid stride, and partial results from blocks sharing a tile are resolved through the workspace.
- *Hybrid Stream-K*: a data-parallel prefix handles complete waves; a streaming tail distributes the remaining tiles' K-iterations across one wave of blocks, eliminating partial-wave waste without applying the streaming overhead to the entire grid.

Both $\hat{q}^l$ and the per-length mode choice are calibrated offline. WISEConv selects $\hat{q}^l$ from a small catalog $\mathcal{Q}$ by profiling each configuration at the full worklist $\mathcal{W}^l_{\text{full}}$ (all $|\mathcal{G}^l|$ output positions) and keeping the fastest. Let $t(q, \mathcal{W})$ denote the minimum kernel latency of configuration q on worklist $\mathcal{W}$ over all mode candidates:

$$\hat{q}^l = \arg\min_{q \in \mathcal{Q}} \; t(q, \; \mathcal{W}^l_{\text{full}}). \tag{7}$$

At the full worklist $\eta_{\text{compute}} \approx 1$, so this step essentially maximizes throughput T, ensuring competitiveness with dense execution. The choice of B_M also determines η_{compute} at every other worklist length: the issued slot count is $|C| = B_M \lceil n^l_{\mathcal{W}} / B_M \rceil$, so $\eta_{\text{compute}} = n^l_{\mathcal{W}} / |C|$ can drop as $n^l_{\mathcal{W}}$ shrinks relative to B_M. Fixing B_M per layer is a necessary constraint, but per-shape profiling partially mitigates the cost: a large B_M on a small output grid often produces too few tiles to fill the device even at full worklist, so such layers naturally select a smaller B_M that also limits η_{compute} loss at low activity. The

mode table below compensates for the remaining throughput gap by adapting the K-splitting strategy on-device.

Within $\hat{q}^l$, WISEConv builds a *policy table* P^l indexed by tile count $s = \lceil n^l_{\mathcal{W}}/B_M \rceil \times \lceil C_{out}/B_N \rceil$. Each entry stores a tail-tile count s_{tail} and a split factor S_k: $s_{\text{tail}} = 0$ yields data-parallel execution, $s_{\text{tail}} = s$ yields full Stream-K, and $0 < s_{\text{tail}} < s$ yields hybrid Stream-K with $s{-}s_{\text{tail}}$ data-parallel prefix tiles. WISEConv profiles a set of sampled tile counts and interpolates to fill the remaining entries:

$$P^l[s] = \arg\min_{s_{\text{tail}},\, S_k} \; t(\hat{q}^l,\; s,\; s_{\text{tail}},\; S_k), \tag{8}$$

where the minimization ranges over candidate tail-tile counts and split factors. All candidates process the same output tiles and differ only in how K-iterations are assigned to blocks, so η_{compute} is invariant and minimizing latency directly maximizes $\eta_{\text{compute}}\, T$.

Algorithm 2 presents the hybrid mode, which subsumes the other two: $s_{\text{tail}} = 0$ yields data-parallel (empty tail), while $s_{\text{tail}} = s$ yields full Stream-K (no data-parallel prefix); the policy table selects among the three based on the current tile count. Algorithm 2 omits boundary-tile padding when $n_{\mathcal{W}}$, C_{in}, or C_{out} is not divisible by B_M, B_K, or B_N, respectively; the implementation masks out-of-bounds accesses in these tiles.

Lines 1–4 of Algorithm 2 read $n_{\mathcal{W}}$, derive the tile count, and look up (s_{tail}, S_k) from P^l. The persistent grid first processes complete waves data-parallel (Lines 7–18): each block decodes its position and channel tile indices, reads B_M output coordinates from the worklist $\mathcal{W}$, and reduces the full reduction depth, writing the result directly to the output. The remaining partial-wave tiles enter the Stream-K tail (Lines 21–36): the reduction depth of each tile is partitioned among S_k contributors (the split factor selected by the policy table), and each block reduces its assigned slice into a pre-allocated workspace slot for that tile. Each slot holds a $B_M \times B_N$ partial accumulator in accumulation precision (typically FP32); the workspace $\mathbf{Z}$ is sized to $\max(S_k) \times |\mathbf{y}^l|$ elements per layer, where $\max(S_k)$ is the largest split factor in the policy table P^l. A per-tile completion counter tracks how many contributors have finished; the last block to arrive loads all partial fragments, reduces them, applies the output epilogue, and writes the final result (Lines 33–37). Both write paths guard stores with M_{out} (§5.2). No device-wide barrier is needed; data-parallel tiles bypass the workspace entirely.

Algorithm 2 On-Device Adaptive Hybrid Stream-K

Require: $\mathcal{W}$, M_{out}, P, g_{res}, B_M, B_N, B_K, $\mathbf{x}$, $\mathbf{f}$, k, $\mathbf{Z}$
Ensure: $\mathbf{y}$

1: $n_{\mathcal{W}} \leftarrow$ **read** device-resident worklist length
2: $s_M \leftarrow \lceil n_{\mathcal{W}}/B_M \rceil$; $s_N \leftarrow \lceil C_{out}/B_N \rceil$; $s \leftarrow s_M \times s_N$
3: $(s_{\text{tail}}, S_k) \leftarrow P[s]$ ▷ *policy lookup*
4: $K_{\text{iters}} \leftarrow k^2 \lceil C_{in}/B_K \rceil$
5: ▷ ——————————*Data-parallel prefix*——————————
6: $s_{\text{dp}} \leftarrow s - s_{\text{tail}}$
7: **for** $b = \text{bid},\ \text{bid}{+}g_{\text{res}},\ \ldots < s_{\text{dp}}$ **do**
8: $(b_m, b_n) \leftarrow \textsc{Decode}(b,\ s_N)$
9: $\mathbf{w} \leftarrow \mathcal{W}[b_m B_M \,..\, (b_m{+}1)B_M)$ ▷ *worklist indirection*
10: $\mathbf{Y} \leftarrow \mathbf{0}_{B_M \times B_N}$
11: **for** $j = 0, \ldots, K_{\text{iters}}{-}1$ **do**
12: $(i, c_k) \leftarrow \textsc{Offset}(j)$ ▷ *kernel offset, channel tile*
13: $\mathbf{X} \leftarrow \mathbf{x}[\mathcal{N}_i(\mathbf{w}),\ c_k B_K{:}(c_k{+}1)B_K)$
14: $\mathbf{F} \leftarrow \mathbf{f}_i[c_k B_K{:}(c_k{+}1)B_K,\ b_n B_N{:}(b_n{+}1)B_N)$
15: $\mathbf{Y} \leftarrow \mathbf{Y} + \mathbf{X} \cdot \mathbf{F}$
16: **end for**
17: $\mathbf{y}[\mathbf{w},\ b_n B_N{:}(b_n{+}1)B_N) \leftarrow \mathbf{Y}$ **where** $M_{out}[\mathbf{w}]{=}1$
18: **end for**
19: ▷ ——————————*Stream-K tail*——————————
20: **for** $b = \text{bid},\ \text{bid}{+}g_{\text{res}},\ \ldots < s_{\text{tail}} \times S_k$ **do**
21: $(b_m, b_n, r) \leftarrow \textsc{TailDecode}(b,\ s_N,\ S_k,\ s_{\text{dp}})$
22: $(k_{\text{lo}}, k_{\text{hi}}) \leftarrow \textsc{Partition}(K_{\text{iters}},\ S_k,\ r)$ ▷ *r-th K-slice*
23: $\mathbf{w} \leftarrow \mathcal{W}[b_m B_M \,..\, (b_m{+}1)B_M)$
24: $\mathbf{Y} \leftarrow \mathbf{0}_{B_M \times B_N}$
25: **for** $j = k_{\text{lo}}, \ldots, k_{\text{hi}}{-}1$ **do**
26: $(i, c_k) \leftarrow \textsc{Offset}(j)$
27: $\mathbf{X} \leftarrow \mathbf{x}[\mathcal{N}_i(\mathbf{w}),\ c_k B_K{:}(c_k{+}1)B_K)$
28: $\mathbf{F} \leftarrow \mathbf{f}_i[c_k B_K{:}(c_k{+}1)B_K,\ b_n B_N{:}(b_n{+}1)B_N)$
29: $\mathbf{Y} \leftarrow \mathbf{Y} + \mathbf{X} \cdot \mathbf{F}$
30: **end for**
31: $\mathbf{Z}[b_m, b_n][r] \leftarrow \mathbf{Y}$ ▷ *partial to workspace*
32: **if** $\textsc{LastArriver}(b_m, b_n)$ **then**
33: $\mathbf{Y} \leftarrow \sum_{r'=0}^{S_k - 1} \mathbf{Z}[b_m, b_n][r']$
34: $\mathbf{y}[\mathbf{w},\ b_n B_N{:}(b_n{+}1)B_N) \leftarrow \mathbf{Y}$ **where** $M_{out}[\mathbf{w}]{=}1$
35: **end if**
36: **end for**

6 Implementation

We implement WISEConv as a PyTorch extension with a Python frontend and a CUDA backend written in C++. A model-conversion pass replaces supported modules, preserves their parameters and weights, and statically marks worklist-reuse edges and rebuild points. The compute backend adapts CUTLASS [1] dense tiled implicit-GEMM pipelines for direct worklist consumption, covering both FP32 (SIMT) and FP16 (tensor core) arithmetic with NHWC tensors; the full-workload comparison uses the FP32 datapath, while sensitivity studies additionally exercise the FP16 datapath. The backend covers all convolution variants used by the evaluated models, as well as mask-aware auxiliary operators. The partial-reduction workspace $\mathbf{Z}$ is shared across all layers at the maximum required size and allocated once during initialization. Every backend variant agrees numerically with cuDNN at each active output position.

7 Evaluation

7.1 Experimental Setup

Platforms. We evaluate six GPU platforms. The four discrete GPUs are NVIDIA A100, RTX 4090, RTX 3080, and RTX 4070 Laptop. The embedded platforms are Jetson Xavier NX and AGX Orin; we also measure board energy on both. The discrete platforms use PyTorch 2.1.2, CUDA 12.1, and cuDNN 8.9.2; both Jetson platforms use PyTorch 2.1.0, CUDA 11.4, and cuDNN 8.6. All paths in the full workload comparison use FP32 (SIMT) with TF32 disabled. Sensitivity studies additionally evaluate FP16 (tensor core) on selected platforms. On each platform, all backends run under the same fixed clock and power configuration.

Models, tasks, and mask sources. Our workloads (Table 2) span three vision tasks: FireFlowNet [28] for event-based optical flow on MVSEC [45] with Ev-Conv [8] masks, YOLOv8n and YOLOv8m [19] for detection on MOT16 [25] with DeltaCNN's [30] temporal masks, and DynConv [37] for human pose on MPII [2] with learned spatial gates. For every workload, calibration uses the training split, while all reported results use a disjoint test split with the authors' released weights [19, 28, 37]. Within each workload, we fix inputs, weights, and upstream mask policy; every sparse path receives the same layer masks, isolating the masked-convolution backend.

Table 2. Evaluation workloads.

	FireFlowNet [28]	**YOLOv8n/m** [19]	**DynConv** [37]
Task	Optical flow	Detection	Human pose
Mask source	Ev-Conv [8]	DeltaCNN [30]	DynConv [37]
Dataset	MVSEC [45]	MOT16 [25]	MPII [2]
Input size	256×256	640×640	256×256
Parameters	0.056M	3.16M / 25.9M	6.89M
Eval. samples	196.8K	2,572	2,958

Baselines. We compare WISEConv against three FP32 (SIMT) execution paths: **Dense** (PyTorch/cuDNN [6] with autotuning), **Tile skipping** (DeltaCNN [30] for FireFlowNet and YOLO; DynConv supplies output masks directly), and **Gather-scatter** (DynConv's [37] CUDA backend for pose; MMCV's [5] MaskedConv2d for FireFlowNet and YOLO).

Measurements. We measure per-frame full-model GPU latency over the complete evaluation datasets. CUDA events bracket each frame's inference; after warm-up we run three complete rounds and average per-frame latency across all frames and rounds. The interval includes mask propagation, worklist construction, all network operators, and output update; it excludes input transfer, offline calibration, and one-time initialization. We report η, T, and T_{eff} at the model level by accumulating required and issued work across layers per frame (Eq. 3), then averaging across frames. On the Jetson platforms, we integrate board-level `VDD_IN` power over each frame interval to obtain per-frame energy. For accuracy, we follow the original protocols: AEE for optical flow [8], AP for detection [30], and PCKh for pose [37]. MOT16 AP uses fixed pseudo-labels from a dense YOLO26x teacher [20] because ground-truth detections cover only the training split.

7.2 End-to-End Performance

Latency. WISEConv is the fastest path in all 24 measured workload-platform pairs (Figure 10). Relative to the fastest evaluated competing path in each pair, it achieves a 1.58× geometric-mean speedup; individual-pair speedups range from 1.33× to 2.12×, corresponding to 25.0–52.8% lower latency. Gains span more than two orders of magnitude in absolute latency, from 0.23 ms for FireFlowNet on RTX 4090 to 88.6 ms for YOLOv8m on Xavier NX.

The two existing sparse paths split the remaining 24 pairs between them: dense is fastest in 13 and tile skipping in 11; gather-scatter never wins. The split tracks active ratio: at the YOLO workloads' $\alpha \approx 48$–49%, few tiles are fully inactive, so tile skipping cannot compensate for its lower throughput; on FireFlowNet ($\alpha \approx 25\%$) and DynConv ($\alpha \approx 16\%$), enough tiles are skippable for it to overtake dense. Neither path is uniformly strong, and WISEConv outperforms whichever wins. On the Jetson platforms the lead narrows (geometric mean 1.53× vs. 1.61× on discrete GPUs) but WISEConv remains fastest in all eight pairs.

Across all 24 pairs, WISEConv averages 1.76× the ideal proportional-scaling reference (dashed lines in Figure 10), compared with 2.79× for the per-pair fastest competing path. The gap to ideal tracks active ratio: on the YOLO workloads WISEConv runs at 1.36× ideal, while on DynConv ($\alpha \approx 16\%$) construction and fixed costs widen it to 2.60×.

Energy. WISEConv is the lowest-energy path for every workload on both Jetson platforms (Table 3), reducing energy by 9.3–36.1% relative to the lowest-energy competing path in each pair and by 29.9–67.8% relative to dense execution.

Table 3. Per-frame board energy (J) on the Jetson platforms. Parentheses report each sparse path's change relative to Dense.

GPU	System	FireFlowNet	YOLOv8n	YOLOv8m	DynConv
AGX Orin	Dense	0.19	0.34	1.82	1.01
	Tile skip	0.10 (-47.9%)	0.31 (-8.0%)	1.59 (-12.7%)	0.51 (-49.7%)
	Gather	0.27 (+44.8%)	0.62 (+83.9%)	1.88 (+3.3%)	2.49 (+145.5%)
	WISEConv	**0.08** (-58.3%)	**0.22** (-35.6%)	**1.06** (-42.0%)	**0.33** (-67.8%)
Xavier NX	Dense	0.28	0.49	2.57	1.17
	Tile skip	0.14 (-48.7%)	0.41 (-14.9%)	2.31 (-10.2%)	0.60 (-48.7%)
	Gather	0.38 (+36.4%)	0.79 (+63.0%)	2.90 (+12.7%)	3.08 (+163.3%)
	WISEConv	**0.13** (-53.5%)	**0.34** (-29.9%)	**1.78** (-30.8%)	**0.51** (-56.8%)

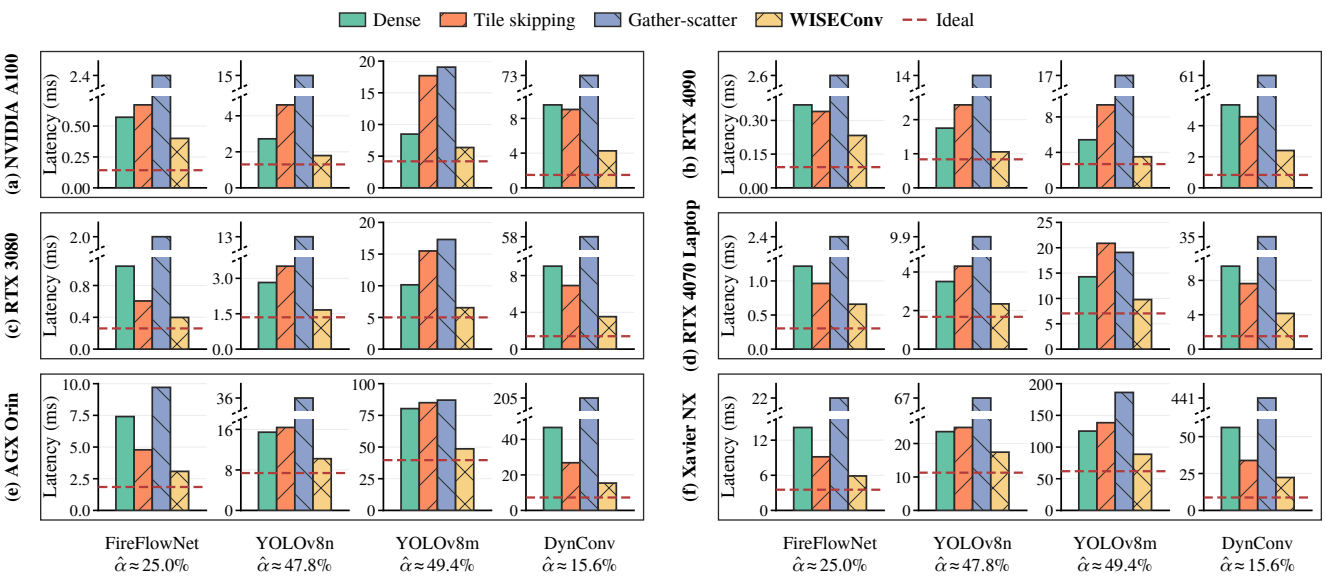


Figure 10. Full-model mean per-frame latency across GPU platforms. Each workload uses an independent y-axis. Labels report the model-level active ratio $\hat{\alpha}$, aggregated as described in §7.1 and averaged across evaluation sequences. Red dashed lines mark ideal proportional scaling, computed by applying each sequence's model-level active ratio to its dense latency before aggregation.

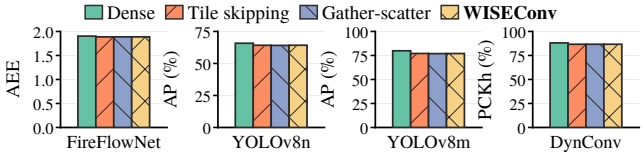


Figure 11. Task accuracy on RTX 3080 under the evaluated upstream sparsity policies, measured by AEE (↓), AP (↑), and PCKh (↑).

Table 4. WISEConv η decomposition and worklist-length statistics on RTX 3080. The n_W percentiles are computed from worklist lengths pooled across all layers and frames.

Workload	$\eta_{\text{construct}}$	η_{compute}	η	n_W		
				P10	P50	P90
FireFlowNet	100.0%	99.8%	99.8%	1,303	15,083	33,880
YOLOv8n	98.6%	99.7%	98.3%	76	937	5,937
YOLOv8m	98.7%	99.5%	98.2%	75	841	6,105
DynConv	100.0%	90.6%	90.6%	3	43	389

Accuracy. WISEConv's latency and energy gains do not alter accuracy. WISEConv matches the other sparse paths on every workload: AEE and PCKh are identical and YOLOv8 AP differs by at most 0.14 points, a residual of floating-point reduction order rather than any difference in covered positions (Figure 11); even where worklist reuse computes a superset of the required positions, the output mask gates writeback so the extra slots do not reach downstream layers. The remaining accuracy gap between sparse and dense, within 4% across all workloads, is set by the upstream mask policy, which WISEConv consumes without modification.

7.3 Microbenchmarks

Effective throughput. A sparse path lowers latency only if it is both selective, captured by the useful-compute ratio η, and fast, captured by raw throughput T (Eq. 4). Figure 12 reports both, normalized to Dense, on RTX 3080 and AGX Orin, and the two existing paths fail on opposite axes. Tile skipping retains the dense tile schedule and so keeps 41–65% of Dense raw throughput, but an active tile computes every position it covers: its useful-compute ratio falls to 0.45–0.72, wasting 28–55% of the slots it issues. Gather-scatter issues exactly the required positions, giving $\eta = 1$, but extraction and writeback collapse its throughput to 2–47% of Dense. WISEConv is the only path that is high on both axes, sustaining $\eta = 0.90$–1.00 and the highest raw throughput among the sparse paths in every workload—42.0–85.1% of Dense on RTX 3080 and 51.3–86.8% on AGX Orin. The two YOLO workloads lie at the upper end because their higher active ratios expose more parallelism and amortize fixed costs; DeltaCNN's throughput on FireFlowNet and DynConv reflects routing very sparse tiles through its fused selective path [30]. Because effective throughput is the product of the two axes, WISEConv reaches 1.38–2.06× the strongest

Table 5. Full-model YOLOv8n latency decomposition on RTX 3080 and AGX Orin.

	System	Tot.	Cons.	Conv.	Elem.	Other
RTX 3080	Dense	2.82	—	2.32(82.0%)	0.39(13.8%)	0.12(4.2%)
	Tile skipping	3.52	—	2.89(82.1%)	0.31(8.9%)	0.31(9.0%)
	Gather	12.60	—	3.82(30.3%)	1.30(10.3%)	7.48(59.4%)
	WISEConv	1.66	0.12(7.5%)	0.99(59.5%)	0.33(19.9%)	0.22(13.1%)
AGX Orin	Dense	15.45	—	11.91(77.1%)	2.49(16.1%)	1.05(6.8%)
	Tile skipping	16.39	—	12.20(74.5%)	1.91(11.7%)	2.27(13.9%)
	Gather	36.09	—	10.25(28.4%)	3.82(10.6%)	22.01(61.0%)
	WISEConv	10.22	0.51(5.0%)	6.06(59.3%)	2.10(20.6%)	1.54(15.1%)

Notes. Entries are milliseconds; percentages are shares of each row's total. Cons. is the WISEConv worklist construction, which the baselines do not have; Other is extraction, scatter, mask, and setup work.

competing effective throughput on RTX 3080 and 1.47–1.87× on AGX Orin.

Table 4 localizes WISEConv's own loss on RTX 3080 through the decomposition of Eq. 5. $\eta_{\text{construct}}$, the fraction of worklist entries that are required positions, stays at 98.6–100.0%: tile-local compaction emits each active coordinate exactly once, and only reuse of a superset worklist gives up the remainder. η_{compute}, the fraction of issued slots that are worklist entries, exceeds 99.5% for FireFlowNet and the two YOLO workloads. DynConv is the exception at 90.6%: its low active ratio and small spatial grids yield exceptionally short worklists (P10, P50, and P90 lengths of 3, 43, and 389), so B_M-sized final tiles are mostly padding. Because its inputs are temporally unrelated, DynConv uses the same exact-length device policy without a temporal activity selector. Even at this worst case the total η is more than double tile skipping's 44.8% on the same workload.

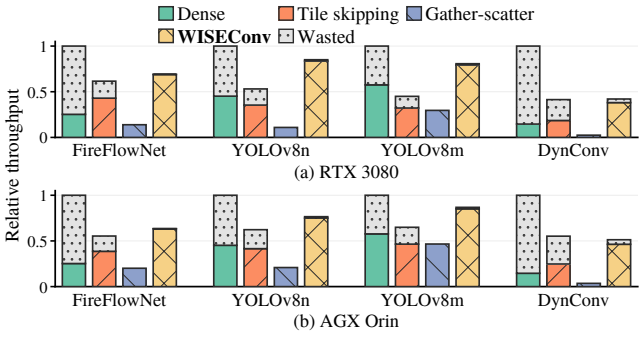


Figure 12. Full-model effective throughput ηT and wasted throughput $(1-\eta)T$ on RTX 3080 and AGX Orin, normalized within each platform to Dense raw throughput.

Latency cost decomposition. We decompose full-model YOLOv8n latency by execution stage on RTX 3080 and AGX Orin (Table 5). On RTX 3080, WISEConv reduces the convolution share from 2.32 to 0.99 ms (2.3×, close to the $1/\alpha \approx 2.2$ proportional target) while adding 0.12 ms of construction, lowering the total from 2.82 to 1.66 ms. On AGX Orin the convolution reduction is 2.0× (11.91 to 6.06 ms) at 0.51 ms of construction, and the total falls from 15.45 to 10.22 ms.

Gather-scatter is the exception: it issues exactly the required positions, so its convolution is the smallest share of any path (30% of its total), but extraction, scatter, and buffer setup cost 7.48 ms, 59% of its total, on RTX 3080. Exact selection alone does not lower latency without a fused datapath.

Memory overhead. Table 6 reports peak live GPU allocation. FireFlowNet, YOLOv8n, and YOLOv8m use temporal mask policies that retain previous-frame activations for differencing, so every sparse path allocates 3–4× Dense. The temporal state dominates the gap, and WISEConv stays within the same tier as tile skipping; gather-scatter is the exception on YOLOv8m, where materializing input patches for each active position adds to the temporal state and pushes it above both other sparse paths. DynConv applies learned spatial gates without temporal state, placing all backends in a narrow 27–34 MiB range. Here WISEConv's worklist and coordinate metadata add only 2.3 MiB over Dense, while gather-scatter is the most compact because DynConv's predominantly pointwise block structure lets intermediates stay in the gathered compact representation [37].

Table 6. Peak live GPU allocation (MiB) on RTX 3080.

System	FireFlowNet	YOLOv8n	YOLOv8m	DynConv
Dense	34.9	76.5	195.1	31.8
Tile skipping	148.4	240.8	752.6	31.8
Gather-scatter	145.1	238.7	824.8	27.4
WISEConv	135.9	246.1	736.0	34.1

7.4 Ablation Study and Sensitivity Analysis

Table 7 isolates each co-designed component by disabling it while holding all others fixed on YOLOv8n; the stage decomposition separates the effect on construction and convolution.

Construction. On RTX 3080, Atomic append and Global order raise construction from 0.12 to 0.43 and 0.54 ms without reducing convolution (0.94 ms throughout), adding 19.1% and 19.6% to the end-to-end latency over Ours; Global order's higher construction reflects its additional kernel passes. No reuse *raises* η to 99.7% ($\eta_{\text{construct}} = 1$ per layer) but increases construction 1.6× (0.12 to 0.19 ms), adding 5.7% over Ours: rebuilding the worklist every layer is more expensive than reusing a slightly loose superset worklist. On AGX Orin the same pattern holds with smaller margins: construction variants add 1–9% over Ours while convolution stays within 0.05 ms, because convolution is a larger share of the total.

Compute. On RTX 3080, DP only raises convolution from 0.94 to 1.01 ms over Ours (+0.07 ms); No hybrid, which retains full Stream-K but not the hybrid mode, recovers most of that

Table 7. YOLOv8n component ablation on RTX 3080 and AGX Orin. *Const.* and *Comp.* group the variants that replace one construction-stage and compute-stage mechanism, respectively; all other settings—weights, inputs, upstream masks, and deployed tile configuration—are held fixed. Cons. and Conv. are the construction- and compute-stage GPU kernel time (ms) profiled in a single eager pass; Tot. is the full-model end-to-end latency (ms), which also includes non-kernel time and therefore need not equal Cons.+Conv.

	Variant	RTX 3080			AGX Orin		
		Cons.	Conv.	Tot.	Cons.	Conv.	Tot.
Const.	Atomic append	0.43	0.94	1.98	1.46	6.21	11.12
	Global order	0.54	0.94	1.99	1.23	6.22	11.04
	No reuse	0.19	0.93	1.76	0.92	6.17	10.33
Comp.	DP only	0.12	1.01	1.73	0.52	6.48	10.38
	No hybrid	0.12	0.96	1.69	0.52	6.33	10.25
	Readback & launch	0.12	0.93	7.61	0.52	6.20	24.33
	Ours	0.12	0.94	1.66	0.52	6.20	10.22

Variants. Atomic append: one global atomic per coordinate instead of per-segment reservation. *Global order*: two-pass stable compaction for full dense-traversal order. *No reuse*: rebuild worklist at every layer. *DP only*: force data-parallel decomposition, disable Stream-K adaptation. *No hybrid*: select between data-parallel and full Stream-K only, disabling the hybrid DP-prefix + Stream-K-tail combination. *Readback & launch*: synchronize worklist length to host and relaunch per operator.

gap (0.96 ms, +0.02 ms over Ours). Readback & launch runs the same kernels as Ours but adds per-operator host synchronization, inflating the total from 1.66 to 7.61 ms (4.58× Ours). On AGX Orin, DP only adds 1.5% end-to-end latency over Ours and readback inflates it to 2.38× Ours.

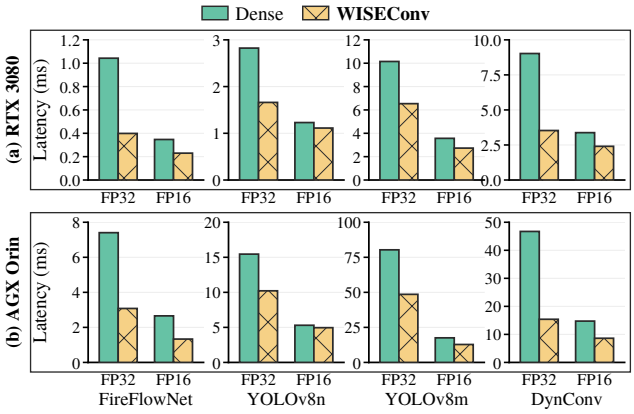


Figure 13. FP32 (SIMT) vs. FP16 (tensor core) full-model latency for Dense and WISEConv across all four workloads on RTX 3080 and AGX Orin.

Tensor core sensitivity. Figure 13 compares FP32 and FP16 execution on RTX 3080 and AGX Orin. The evaluated sparse baselines [5, 30, 37] provide only FP32 paths, so this study compares Dense and WISEConv only. WISEConv retains a 1.07–1.99× speedup over Dense under FP16 tensor cores across all eight workload-platform pairs, narrowing from 1.51–3.04× under FP32. The narrowing is expected: tensor cores reduce the convolution time that WISEConv saves by skipping, shrinking the absolute saving while non-convolution costs (construction, elementwise, mask processing) do not shrink proportionally. The two YOLO workloads narrow the most (1.51–1.70× under FP32 to 1.07–1.37× under FP16) because their high active ratio ($\alpha \approx 48$–49%) already limits the sparsity gain; within the two, YOLOv8n narrows further because its smaller model makes non-convolution costs a larger share of the total under FP16.

Table 8. YOLOv8n latency decomposition under stream-level batching on RTX 3080. Each sequence is split into disjoint temporal streams; each batch combines one current frame per stream and advances their states independently. Stage definitions, units, and cell format follow Table 5.

	System	Tot.	Cons.	Conv.	Elem.	Other
Batch size=1	Dense	2.82	—	2.32(82.0%)	0.39(13.8%)	0.12(4.2%)
	Tile skipping	3.52	—	2.89(82.1%)	0.31(8.9%)	0.31(9.0%)
	Gather	12.60	—	3.82(30.3%)	1.30(10.3%)	7.48(59.4%)
	WISEConv	1.66	0.12(7.5%)	0.99(59.5%)	0.33(19.9%)	0.22(13.1%)
Batch size=4	Dense	1.82	—	1.38(75.4%)	0.31(17.0%)	0.14(7.6%)
	Tile skipping	1.69	—	1.17(69.2%)	0.27(15.7%)	0.26(15.1%)
	Gather	6.44	—	1.92(29.8%)	0.78(12.1%)	3.74(58.1%)
	WISEConv	1.02	0.03(3.1%)	0.58(56.6%)	0.23(22.8%)	0.18(17.6%)
Batch size=8	Dense	1.69	—	1.20(70.9%)	0.33(19.5%)	0.16(9.6%)
	Tile skipping	1.43	—	0.92(64.0%)	0.26(18.3%)	0.25(17.7%)
	Gather	4.06	—	1.14(28.0%)	0.51(12.6%)	2.41(59.4%)
	WISEConv	0.90	0.02(1.7%)	0.49(54.3%)	0.22(24.0%)	0.18(19.9%)

Stream-level batching. Table 8 reports per-frame latency decomposition for YOLOv8n on RTX 3080 at batch sizes 1, 4, and 8. WISEConv remains fastest at every batch size, but its lead over the fastest competing path narrows from 1.70× at batch 1 to 1.59× at batch 8. Tile skipping benefits most: from batch 1 to 8, its per-frame convolution drops from 2.89 to 0.92 ms, compared with 0.99 to 0.49 ms for WISEConv, because batching adds independent tiles that fill otherwise idle SMs [30], the same occupancy that WISEConv's adaptive decomposition already provides at batch 1. Tile skipping accordingly overtakes dense at batch 4 and stays the fastest competing path at batch 8. Gather-scatter's extraction overhead does not amortize: *Other* still accounts for 59.4% of its per-frame latency at batch 8.

7.5 Discussion

WISEConv consumes rather than generates masks, so its gains are conditional on the upstream policy and its activity distribution; it guarantees coverage of requested outputs but does not improve mask quality. When the active ratio is

high enough, construction overhead and the gap between the worklist kernel and cuDNN leave little room for a net gain, as the YOLO workloads' narrower speedups already suggest.

The adaptive decomposition is calibrated per convolution shape and device; results are cached and reused across models that share the same shapes, but a new shape or device requires reprofiling. Because calibration selects only among decomposition modes, it affects performance but not coverage or correctness. The current calibration policy is simple; a faster or more effective one is a natural direction for future work.

The full workload comparison uses FP32 (SIMT) because the evaluated sparse baselines [5, 30, 37] provide only FP32 paths. The sensitivity study confirms that WISEConv retains end-to-end gains under FP16 tensor cores on both evaluated platforms across all four workloads. Extending the worklist compute stage to native tensor-core reduction remains open. Stream-K performs FP32 partial-sum reduction through an external workspace (§5.3), which is necessary for numerical precision but adds overhead that, for the short worklists where split-K is most needed, can outweigh the tensor-core arithmetic savings. A broader study across GPU architectures remains for future work.

8 Conclusion

We presented WISEConv, a masked-convolution runtime that replaces the dense tiled pipeline's contiguous coordinate stream with an active-output worklist, computing almost exclusively the positions an upstream mask requires while retaining the dense tensor layout and the regular per-position reduction. The construction and consumption stages are co-designed to secure both useful-compute ratio and throughput; our ablation confirms that disabling any single mechanism degrades one axis without improving the other. The resulting higher effective throughput lets WISEConv convert upstream mask sparsity into proportional latency and energy reduction for the latency-critical vision tasks where masked convolution applies.

References

[1] 2026. CUTLASS: CUDA Templates and Python DSLs for High-Performance Linear Algebra. https://github.com/NVIDIA/cutlass

[2] Mykhaylo Andriluka, Leonid Pishchulin, Peter Gehler, and Bernt Schiele. 2014. 2D Human Pose Estimation: New Benchmark and State of the Art Analysis. In *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 3686–3693.

[3] Rik J. Bouwmeester, Federico Paredes-Vallés, and Guido C. H. E. de Croon. 2023. NanoFlowNet: Real-time Dense Optical Flow on a Nano Quadcopter. In *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*. 1996–2003.

[4] Lukas Cavigelli, Philippe Degen, and Luca Benini. 2017. CBinfer: Change-Based Inference for Convolutional Neural Networks on Video Data. In *Proc. Int. Conf. Distrib. Smart Cameras (ICDSC)*. 1–8.

[5] Kai Chen, Jiaqi Wang, Jiangmiao Pang, Yuhang Cao, Yu Xiong, Xiaoxiao Li, Shuyang Sun, Wansen Feng, Ziwei Liu, Jiarui Xu, Zheng Zhang, Dazhi Cheng, Chenchen Zhu, Tianheng Cheng, Qijie Zhao, Buyu Li, Xin Lu, Rui Zhu, Yue Wu, Jifeng Dai, Jingdong Wang, Jianping Shi, Wanli Ouyang, Chen Change Loy, and Dahua Lin. 2019. MMDetection: Open MMLab Detection Toolbox and Benchmark. *CoRR* abs/1906.07155 (2019).

[6] Sharan Chetlur, Cliff Woolley, Philippe Vandermersch, Jonathan Cohen, John Tran, Bryan Catanzaro, and Evan Shelhamer. 2014. cuDNN: Efficient Primitives for Deep Learning. *CoRR* abs/1410.0759 (2014).

[7] Christopher B. Choy, JunYoung Gwak, and Silvio Savarese. 2019. 4D Spatio-Temporal ConvNets: Minkowski Convolutional Neural Networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.

[8] Sankeerth Durvasula, Yushi Guan, and Nandita Vijaykumar. 2023. EvConv: Fast CNN Inference on Event Camera Inputs for High-Speed Robot Perception. *IEEE Robotics Autom. Lett.* 8, 6 (2023), 3174–3181.

[9] Davide Falanga, Philipp Foehn, Peng Lu, and Davide Scaramuzza. 2018. PAMPC: Perception-Aware Model Predictive Control for Quadrotors. In *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems, IROS 2018, Madrid, Spain, October 1-5, 2018*. IEEE, 1–8. doi:10.1109/IROS.2018.8593739

[10] Davide Falanga, Suseong Kim, and Davide Scaramuzza. 2019. How Fast Is Too Fast? The Role of Perception Latency in High-Speed Sense and Avoid. *IEEE Robotics Autom. Lett.* 4, 2 (2019), 1884–1891. doi:10.1109/LRA.2019.2898117

[11] Davide Falanga, Kevin Kleber, and Davide Scaramuzza. 2020. Dynamic obstacle avoidance for quadrotors with event cameras. *Sci. Robotics* 5, 40 (2020), 9712. doi:10.1126/SCIROBOTICS.AAZ9712

[12] Ruibo Fan, Wei Wang, and Xiaowen Chu. 2024. DTC-SpMM: Bridging the Gap in Accelerating General Sparse Matrix Multiplication with Tensor Cores. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3, ASPLOS 2024, La Jolla, CA, USA, 27 April 2024- 1 May 2024*, Rajiv Gupta, Nael B. Abu-Ghazaleh, Madan Musuvathi, and Dan Tsafrir (Eds.). ACM, 253–267. doi:10.1145/3620666.3651378

[13] Michael Figurnov, Maxwell D. Collins, Yukun Zhu, Li Zhang, Jonathan Huang, Dmitry P. Vetrov, and Ruslan Salakhutdinov. 2017. Spatially Adaptive Computation Time for Residual Networks. In *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 1790–1799.

[14] Trevor Gale, Matei Zaharia, Cliff Young, and Erich Elsen. 2020. Sparse GPU Kernels for Deep Learning. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*.

[15] Yizhao Gao, Baoheng Zhang, Xiaojuan Qi, and Hayden Kwok-Hay So. 2023. DPACS: Hardware Accelerated Dynamic Neural Network Pruning through Algorithm-Architecture Co-design. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS 2023, Vancouver, BC, Canada, March 25-29, 2023*, Tor M. Aamodt, Natalie D. Enright Jerger, and Michael M. Swift (Eds.). ACM, 237–251. doi:10.1145/3575693.3575728

[16] Benjamin Graham, Martin Engelcke, and Laurens van der Maaten. 2018. 3D Semantic Segmentation with Submanifold Sparse Convolutional Networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.

[17] Amirhossein Habibian, Davide Abati, Taco S. Cohen, and Babak Ehteshami Bejnordi. 2021. Skip-Convolutions for Efficient Video Processing. In *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 2695–2704.

[18] Drew Hanover, Antonio Loquercio, Leonard Bauersfeld, Angel Romero, Robert Penicka, Yunlong Song, Giovanni Cioffi, Elia Kaufmann, and Davide Scaramuzza. 2024. Autonomous Drone Racing: A Survey. *IEEE Trans. Robotics* 40 (2024), 3044–3067. doi:10.1109/TRO.2024.3400838

[19] Glenn Jocher, Ayush Chaurasia, and Jing Qiu. 2023. Ultralytics YOLOv8. https://github.com/ultralytics/ultralytics

[20] Glenn Jocher, Jing Qiu, Mengyu Liu, Shuai Lyu, Fatih Cagatay Akyon, and Muhammet Esat Kalfaoglu. 2026. Ultralytics YOLO26: Unified Real-Time End-to-End Vision Models. *CoRR* abs/2606.03748 (2026).

[21] Fredrik Kjolstad, Shoaib Kamil, Stephen Chou, David Lugato, and Saman P. Amarasinghe. 2017. The Tensor Algebra Compiler. *Proc. ACM Program. Lang.* 1, OOPSLA (2017).

[22] Fanrong Li, Gang Li, Xiangyu He, and Jian Cheng. 2021. Dynamic Dual Gating Neural Networks. In *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*. 5310–5319.

[23] Renyuan Liu, Yuyang Leng, Shilei Tian, Shaohan Hu, Chun-Fu (Richard) Chen, and Shuochao Yao. 2024. DynaSpa: Exploiting Spatial Sparsity for Efficient Dynamic DNN Inference on Devices. In *Proceedings of the 22nd ACM Conference on Embedded Networked Sensor Systems, SenSys 2024, Hangzhou, China, November 4-7, 2024*. ACM, 422–435. doi:10.1145/3666025.3699348

[24] Nico Messikommer, Daniel Gehrig, Antonio Loquercio, and Davide Scaramuzza. 2020. Event-Based Asynchronous Sparse Convolutional Networks. In *Proc. Eur. Conf. Comput. Vis. (ECCV)*. 415–431.

[25] Anton Milan, Laura Leal-Taixé, Ian D. Reid, Stefan Roth, and Konrad Schindler. 2016. MOT16: A Benchmark for Multi-Object Tracking. *CoRR* abs/1603.00831 (2016).

[26] Asit K. Mishra, Jorge Albericio Latorre, Jeff Pool, Darko Stosic, Dusan Stosic, Ganesh Venkatesh, Chong Yu, and Paulius Micikevicius. 2021. Accelerating Sparse Deep Neural Networks. *CoRR* abs/2104.08378 (2021).

[27] Muhammad Osama, Duane Merrill, Cris Cecka, Michael Garland, and John D. Owens. 2023. Stream-K: Work-centric Parallel Decomposition for Dense Matrix-Matrix Multiplication on the GPU. In *Proceedings of the 28th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming (PPoPP)*. doi:10.1145/3572848.3577479

[28] Federico Paredes-Vallés and Guido C. H. E. de Croon. 2021. Back to Event Basics: Self-Supervised Learning of Image Reconstruction for Event Cameras via Photometric Constancy. In *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 3445–3454.

[29] Mathias Parger, Chengcheng Tang, Thomas Neff, Christopher D. Twigg, Cem Keskin, Robert Wang, and Markus Steinberger. 2023. MotionDeltaCNN: Sparse CNN Inference of Frame Differences in Moving Camera Videos with Spherical Buffers and Padded Convolutions. In *IEEE/CVF International Conference on Computer Vision, ICCV 2023, Paris, France, October 1-6, 2023*. 17246–17255.

[30] Mathias Parger, Chengcheng Tang, Christopher D. Twigg, Cem Keskin, Robert Wang, and Markus Steinberger. 2022. DeltaCNN: End-to-End CNN Inference of Sparse Frame Differences in Videos. In *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 12487–12496.

[31] Hongwu Peng, Xi Xie, Kaustubh Shivdikar, Md Amit Hasan, Jiahui Zhao, Shaoyi Huang, Omer Khan, David R. Kaeli, and Caiwen Ding. 2024. MaxK-GNN: Extremely Fast GPU Kernel Design for Accelerating Graph Neural Networks Training. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS 2024, La Jolla, CA, USA, 27 April 2024- 1 May 2024*, Rajiv Gupta, Nael B. Abu-Ghazaleh, Madan Musuvathi, and Dan Tsafrir (Eds.). ACM, 683–698. doi:10.1145/3620665.3640426

[32] Mengye Ren, Andrei Pokrovsky, Bin Yang, and Raquel Urtasun. 2018. SBNet: Sparse Blocks Network for Fast Inference. In *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 8711–8720.

[33] Spconv Contributors. 2022. Spconv: Spatially Sparse Convolution Library. https://github.com/traveller59/spconv.

[34] Martin Sundermeyer, Arsalan Mousavian, Rudolph Triebel, and Dieter Fox. 2021. Contact-GraspNet: Efficient 6-DoF Grasp Generation in Cluttered Scenes. In *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*. 13438–13444.

[35] Haotian Tang, Zhijian Liu, Xiuyu Li, Yujun Lin, and Song Han. 2022. TorchSparse: Efficient Point Cloud Inference Engine. In *Proceedings of Machine Learning and Systems (MLSys)*.

[36] Haotian Tang, Shang Yang, Zhijian Liu, Ke Hong, Zhongming Yu, Xiuyu Li, Guohao Dai, Yu Wang, and Song Han. 2023. TorchSparse++: Efficient Training and Inference Framework for Sparse Convolution on GPUs. In *Proc. IEEE/ACM Int. Symp. Microarchitecture (MICRO)*. 225–239.

[37] Thomas Verelst and Tinne Tuytelaars. 2020. Dynamic Convolutions: Exploiting Spatial Sparsity for Faster Inference. In *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 2320–2329.

[38] Chen Wang, Danfei Xu, Yuke Zhu, Roberto Martin Martin, Cewu Lu, Li Fei-Fei, and Silvio Savarese. 2019. DenseFusion: 6D Object Pose Estimation by Iterative Dense Fusion. In *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 3343–3352.

[39] Kunyun Wang, Shuo Yang, Jieru Zhao, Wenchao Ding, Quan Chen, Jingwen Leng, and Minyi Guo. 2025. SparseTem: Boosting the Efficiency of CNN-Based Video Encoders by Exploiting Temporal Continuity. In *Advanced Parallel Processing Technologies - 16th International Symposium, APPT 2025, Athens, Greece, July 13-16, 2025, Proceedings*. 246–256.

[40] Diana Wofk, Fangchang Ma, Tien-Ju Yang, Sertac Karaman, and Vivienne Sze. 2019. FastDepth: Fast Monocular Depth Estimation on Embedded Systems. In *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*. 6101–6108.

[41] Zhenda Xie, Zheng Zhang, Xizhou Zhu, Gao Huang, and Stephen Lin. 2020. Spatially Adaptive Inference with Stochastic Feature Sampling and Interpolation. In *Proc. Eur. Conf. Comput. Vis. (ECCV)*. 531–548.

[42] Jiacheng Yang, Christina Giannoula, Jun Wu, Mostafa Elhoushi, James Gleeson, and Gennady Pekhimenko. 2024. Minuet: Accelerating 3D Sparse Convolutions on GPUs. In *Proceedings of the Nineteenth European Conference on Computer Systems, EuroSys 2024, Athens, Greece, April 22-25, 2024*. 786–802.

[43] Zihao Ye, Ruihang Lai, Junru Shao, Tianqi Chen, and Luis Ceze. 2023. SparseTIR: Composable Abstractions for Sparse Compilation in Deep Learning. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*.

[44] Changqian Yu, Jingbo Wang, Chao Peng, Changxin Gao, Gang Yu, and Nong Sang. 2018. BiSeNet: Bilateral Segmentation Network for Real-Time Semantic Segmentation. In *Proc. Eur. Conf. Comput. Vis. (ECCV)*. 334–349.

[45] Alex Zihao Zhu, Dinesh Thakur, Tolga Özaslan, Bernd Pfrommer, Vijay Kumar, and Kostas Daniilidis. 2018. The Multi Vehicle Stereo Event Camera Dataset: An Event Camera Dataset for 3D Perception. *CoRR* abs/1801.10202 (2018).